

ECOM EXPERTS

Ecom agenteseo

Manual de usuario, tecnico y de desarrollo

ecom_agenteseo · 08/09/2026

Contenido

Contexto

Objetivo

Estrategia de Implementación (Hook: displayAdminProductsExtra)

1. Estructura de la Tabla

2. Código del Hook (En ecom_agenteseo.php)

3. Guardado (Hook: actionProductUpdate)

Instrucción de Comportamiento

name: create_product_tab description: Instrucciones para que el Agente aprenda a crear Pestañas (Tabs) en la ficha de producto de PrestaShop.

SKILL: Gestión de Pestañas en Producto (AdminProduct)

Contexto

El usuario quiere centralizar todos los campos personalizados (SEO, Marketing, Agentes, etc.) en una **ÚNICA PESTAÑA** dentro de la ficha de edición de producto en el Backoffice.

Objetivo

Cuando necesites guardar información extra de un producto (ej: meta keywords generadas, análisis de sentimiento, notas del agente), NO crees tablas sueltas ni hooks dispersos. Debes: 1. Crear (si no existe) una tabla `ps_ecom_agenteseo_product` vinculada por `id_product`. 2. Usar el hook `displayAdminProductsExtra` (PS 1.7/8 Legacy) o el nuevo sistema de hooks de Symfony para inyectar una **Pestaña Nueva**. 3. El nombre de la pestaña debe ser "SEO Agent Data".

Estrategia de Implementación (Hook: displayAdminProductsExtra)

Este hook permite añadir una pestaña al final de la ficha de producto clásica o módulos laterales en la nueva.

1. Estructura de la Tabla

Si vas a guardar datos del producto, asegura esta tabla:

```
CREATE TABLE IF NOT EXISTS `ps_ecom_agenteseo_product` (  
  `id_ecom_agenteseo_product` int(11) NOT NULL AUTO_INCREMENT,  
  `id_product` int(11) NOT NULL,  
  `id_shop` int(11) NOT NULL DEFAULT '1',  
  `ai_audit_json` text, -- Para guardar análisis completos  
  `custom_keywords` text,  
  `date_upd` datetime,  
  PRIMARY KEY (`id_ecom_agenteseo_product`),  
  UNIQUE KEY `idx_product` (`id_product`, `id_shop`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

2. Código del Hook (En ecom_agenteseo.php)

Debes registrar `displayAdminProductsExtra` y renderizar un TPL.

```
public function hookDisplayAdminProductsExtra($params) {
    $id_product = (int)$params['id_product'];
    // Cargar datos de tu tabla
    // Assign smarty
    return $this->display(__FILE__, 'views/templates/admin/product_tab.tpl');
}
```

3. Guardado (Hook: actionProductUpdate)

Para guardar los datos automáticamente cuando el usuario pulse "Guardar" en el producto.

```
public function hookActionProductUpdate($params) {
    $id_product = (int)$params['id_product'];
    if (Tools::isSubmit('ecom_agent_keywords')) {
        // SQL Insert/Update a ps_ecom_agenteseo_product
    }
}
```

Instrucción de Comportamiento

Si el usuario te pide "Añade un campo para X al producto": 1. Revisa si la tabla `ps_ecom_agenteseo_product` tiene esa columna. Si no, haz un `ALTER TABLE`. 2. Modifica el archivo `views/templates/admin/product_tab.tpl` para añadir el input. 3. Asegura que el `hookActionProductUpdate` procese ese nuevo campo. 4. **IMPORTANTE:** Usa siempre esta pestaña unificada. No crees pestañas nuevas para cada chorrada.